Preprocessing: import modules, read the image, and create the output directory for pictures.

```python
import cv2
import numpy as np
import os
import matplotlib.pyplot as plt

# read image
path = r"C:\ntu_csie\1141\cv\lena.bmp"
img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
h, w = img.shape

# create output dir
output_dir = r"C:\ntu_csie\1141\cv\output\hw7"
os.makedirs(output_dir, exist_ok=True)
```

Binarize and downsample the image.

```python
# binarize
bi = np.zeros_like(img, dtype=int)
for i in range(h):
    for j in range(w):
        bi[i, j] = 1 if img[i, j] >= 128 else 0

# downsample
down_h, down_w = h // 8, w // 8
down = np.zeros((down_h, down_w), dtype=int)
for i in range(down_h):
    for j in range(down_w):
        down[i,j] = bi[i*8, j*8]
```

Define h() and yokoi() as described in CV1_CH6_2025.pptx page 60.

6.2.6 Connected Shrink Operator

- For 4-connectivity

$$h(b,c,d,e) = \begin{cases} 1 & \text{if } b = c \wedge (b \neq d \vee b \neq e) \\ 0 & \text{otherwise} \end{cases}$$

- For 8-connectivity

$$h(b,c,d,e) = \begin{cases} 1 & \text{if } b \neq c \wedge (b = d \vee b = e) \\ 0 & \text{otherwise} \end{cases}$$

Hint: *b = c 且 (b ≠ d 或 b ≠ e) 則為 1。*
其他情況為 0。
Hint: *b ≠ c 且 (b = d 或 b = e) 則為 1。*
其他情況為 0。

	e	d
	b	c

Corner Neighborhood (for corresponding x_i)

- $a_1 = h(x_0, x_1, x_6, x_2)$
- $a_2 = h(x_0, x_2, x_7, x_3)$
- $a_3 = h(x_0, x_3, x_8, x_4)$
- $a_4 = h(x_0, x_4, x_5, x_1)$

a_1

	x_2	x_6
	x_0	x_1

a_2

x_7	x_2	
x_3	x_0	

a_3

x_3	x_0	
x_8	x_4	

a_4

	x_0	x_1
	x_4	x_5

- $output = f(a_1, a_2, a_3, a_4, x_0) = \begin{cases} g & \text{if } exactly\ one\ of\ a_1, a_2, a_3, a_4 = 1 \\ x_0 & \text{otherwise} \end{cases}$

Hint: 只有一個為 1 則令為 g (background)
Hint: 否則為原樣

```python
def h(b, c, d, e):
    if b == c and (d != b or e != b):
        return 1
    else:
        return 0

def valid(x, y, h, w):
    return 0 <= x < h and 0 <= y < w

def f(img, x, y):
    if img[x, y] == 0:
        return 0

    row, col = img.shape
    x0 = img[x, y]
    x1 = img[x, y+1] if valid(x, y+1, row, col) else 0
    x2 = img[x-1, y] if valid(x-1, y, row, col) else 0
    x3 = img[x, y-1] if valid(x, y-1, row, col) else 0
    x4 = img[x+1, y] if valid(x+1, y, row, col) else 0
    x5 = img[x+1, y+1] if valid(x+1, y+1, row, col) else 0
    x6 = img[x-1, y+1] if valid(x-1, y+1, row, col) else 0
    x7 = img[x-1, y-1] if valid(x-1, y-1, row, col) else 0
    x8 = img[x+1, y-1] if valid(x+1, y-1, row, col) else 0

    a1 = h(x0, x1, x6, x2)
    a2 = h(x0, x2, x7, x3)
    a3 = h(x0, x3, x8, x4)
    a4 = h(x0, x4, x5, x1)

    if (a1 + a2 + a3 + a4) == 1:
        return 0
    else:
        return 1
```

Define the functions that mark interior/boarder, mark pair relationship, and do connected shrink, respectively.

```python
def mark_b_i(img):
    row, col = img.shape
    mark = np.full((row, col), ' ', dtype='<U1')
    for x in range(row):
        for y in range(col):
            if img[x, y] == 0:
                continue
            cnt = 0
            for (r, c) in [(x-1, y), (x+1, y), (x, y-1), (x, y+1)]:
                if valid(r, c, row, col) and img[r, c] == 1:
                    cnt += 1
            mark[x, y] = 'i' if cnt == 4 else 'b'
    return mark
```

```python
def pair_relationship(img):
    row, col = img.shape
    pq = np.full((row, col), ' ', dtype='<U1')
    for x in range(row):
        for y in range(col):
            if img[x, y] == ' ':
                continue
            if img[x, y] == 'b':
                has_i = False
                for (r, c) in [(x-1, y), (x+1, y), (x, y-1), (x, y+1)]:
                    if valid(r, c, row, col) and img[r, c] == 'i':
                        has_i = True
                        break
                pq[x, y] = 'p' if has_i else 'q'
            else:
                pq[x, y] = 'q'
    return pq
```

```python
def connected_shrink(img, mark_pq):
    row, col = img.shape
    out = img.copy()
    removed = 0
    for x in range(row):
        for y in range(col):
            if img[x, y] == 0:
                continue
            if mark_pq[x, y] != 'p':
                continue
            if f(img, x, y) == 0:
                out[x, y] = 0
                removed += 1
    return out, removed
```

Recursively do the shrinking until no pixel changed, and then save the image.

```python
cur = down.copy()
while True:
    mark_ib = mark_b_i(cur)
    mark_pq = pair_relationship(mark_ib)
    new_img, removed = connected_shrink(cur, mark_pq)
    cur = new_img
    if removed == 0:
        break

thin = (cur * 255).astype(np.uint8)
out_img_path = os.path.join(output_dir, "thin.png")
cv2.imwrite(out_img_path, thin)
print(f"Image saved: {out_img_path}")
```